Python Engineering - Classes & Objects

Entidad, clase, objeto, atributos, metodos, modulos y self

Esta guia explica de forma sencilla las palabras basicas de programacion orientada a objetos. La idea no es memorizar definiciones, sino entender como encajan unas con otras.

1. Entidad: la cosa que queremos representar

Una **entidad** es una idea del problema que queremos tratar como una cosa con significado propio. Puede ser un avion, un satelite, un sensor, una IMU o, en EX21, un estado de movimiento vertical.

Ejemplo conceptual: queremos representar algo que tenga altura h, velocidad v, tiempo t y gravedad g.

2. Clase: la definicion general

La **clase** describe como seran los objetos de ese tipo: que datos guardaran y que operaciones podran realizar.

```
class VerticalState:
    ...
```

Una clase es parecida a un plano de diseno. Todavia no representa un estado concreto de 100 m y 20 m/s; solo define como sera cualquier VerticalState.

3. Objeto: una realizacion concreta de la clase

Un **objeto** o **instancia** es una realizacion concreta creada a partir de la clase.

```
earth_state = VerticalState(
    h=100.0,
    v=20.0,
    g=9.81
)

moon_state = VerticalState(
    h=100.0,
    v=20.0,
    g=1.62
)
```

earth_state y moon_state pertenecen a la misma clase, pero son objetos diferentes y guardan estados independientes.

4. Atributos: los datos que guarda el objeto

Los **atributos** son datos asociados a un objeto. En nuestro caso:

```
earth_state.h
earth_state.v
earth_state.t
earth_state.g
```

Podemos pensar que los atributos responden a la pregunta: **que sabe o que contiene este objeto?**

5. Metodos: lo que el objeto sabe hacer

Un **metodo** es una funcion definida dentro de una clase. Se parece a una funcion normal, pero esta pensada para trabajar con un objeto de esa clase.

```
def specific_energy(self):
    return self.g * self.h + 0.5 * self.v**2

def euler_step(self, dt):
    self.h = self.h + self.v * dt
    self.v = self.v - self.g * dt
    self.t = self.t + dt
```

Los atributos son lo que el objeto contiene. Los metodos son lo que el objeto puede hacer.

6. Que significa self?

self significa este objeto concreto. Cuando un mismo metodo puede ser usado por varios objetos, self indica sobre cual de ellos se esta trabajando en ese momento.

```
earth_state.specific_energy()
# inside the method: self -> earth_state

moon_state.specific_energy()
# inside the method: self -> moon_state
```

Por eso self.h significa la altura del objeto actual; self.g significa la gravedad almacenada en ese objeto.

7. h frente a self.h

```
def __init__(self, h, v, t=0.0, g=9.81):
    self.h = h
```

h es un parametro recibido durante esa llamada. `self.h` es el atributo que queda guardado dentro del objeto.

El lado derecho es el valor que entra; el lado izquierdo es donde lo almacenamos.

8. __init__: preparar el objeto

__init__ es el metodo que inicializa el estado del objeto cuando lo creamos.

```
earth_state = VerticalState(h=100.0, v=20.0)

# approximately:
# create object
# call __init__
# store h, v, t, g
# return the prepared object reference
```

9. Modulo: el archivo .py que organiza codigo

Un **modulo** es, de forma sencilla, un archivo .py que contiene codigo reutilizable. Un modulo puede contener funciones, clases, constantes y otras definiciones.

```
vertical_state.py
    contains -> class VerticalState

main.py
    imports -> VerticalState
```

Modulo, clase y objeto son niveles diferentes. Un modulo puede contener una clase; una clase puede crear objetos.

10. Jerarquia mental completa

```
ENTITY
vertical-motion state
        |
        v
MODULE: vertical_state.py
        |
        contains
        v
CLASS: VerticalState
        |
        creates
        v
OBJECTS: earth_state, moon_state
        |
        contain
        v
ATTRIBUTES: h, v, t, g
        |
        and use
```

```
        v
METHODS: specific_energy(), euler_step(), ...
        |
        self means
        v
'this particular object'
```

11. Ejemplo aeronautico

```
class Aircraft:
    # attributes:
    # mass, position, velocity, fuel, attitude
    # methods:
    # calculate_lift(), calculate_drag(), burn_fuel()

aircraft_1 = Aircraft(...)
aircraft_2 = Aircraft(...)
```

Si llamamos aircraft_1.burn_fuel(), cambia el estado de aircraft_1, no el de aircraft_2. La clase es comun; cada objeto mantiene sus propios atributos.

Resumen final

Concepto	Idea sencilla
Entidad	La cosa del problema que queremos representar
Modulo	Archivo .py que organiza codigo
Clase	Definicion general / plano
Objeto	Instancia concreta de una clase
Atributo	Dato guardado en el objeto
Metodo	Funcion de la clase que trabaja con el objeto
self	Referencia al objeto concreto actual
__init__	Inicializa los atributos del objeto